

LLM Usage and Code Understanding Report

Project 2 — Student Report Template

Student ID	20211163
Name	Song WooYoung
Date	2026-06-06 (SAT)
LLM Tool(s) Used	None

Purpose: Verify that you understand your own code, can explain the algorithms, and can describe how you debugged, modified, and verified your implementation.
Do not paste raw AI chat logs. Summarize reasoning and decisions in your own words.
You are fully responsible for the correctness and understanding of your submitted code.

Problem 1 — Part 1

Conveyor Belt Sorter

A Problem Encountered or Difficult Part

There were two main problems that I have to handle. First was that unified int random generator and second one is where to accumulate the inverse num in merge sort.

First is that whatever I did modify the code I cannot get the example array offered by PDF.

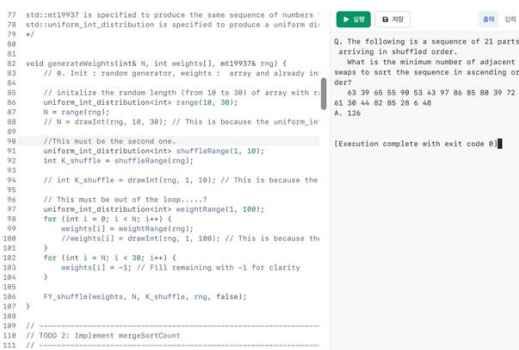
So, I had searched for 30 minutes for figuring out the reason and found out that despite the consistent random machine mt19937, Unified random int generator was developed differently by compiled builder. To prove that I execute the exactly same code on the myCompiler and IDEone and got the result of what I found. Even the identical codes are put, the result deviated TOO far. Later, after asking this problem, TA thankfully accepted my opinion and The radom array generator is changed from using Unified ~~ to drawInt()

Second, I have ever learned the MergeSort in Algorithm class, So, it was not that hard to implement its code (and even the implementation is SO FAMOUS that without AI, it is possible to refer the code on internet). However, this case requires that “Not modifying and only counting the inverse Number of inverse orders”. Not modifying can be resolved using the copy of the array but, when counting the number of Inverse, I mistakenly did count the number when during merging, I did `inverse_num++` (specifics will be explained next section), which was not the case. It would be much clear if we think of the situation where we merge 8 integers into one array, which contains 1,7,9,10 / 2,3,4,5. At first step, It is for sure that 1 is smaller than 2 so, we put 1 and move the left hand cursor to next integer, 7. Here, I did miscount the number of inverse. When thinking of 2 is smaller than 2, 2 have to pass by 7,9 and 10!!!. However, I did only `inv_num++`..

After changing this code, all become what we expect.

B How You Solved It — Evidence & Process

<https://stackoverflow.com/questions/70841470/c-pseudo-random-number-generator-not-dependent-on-kernel-version>



```

77 std::mt19937 is specified to produce the same sequence of numbers
78 std::uniform_int_distribution is specified to produce a uniform di
79 *)
80
81
82 void generateWeights(int n, int weights[], mt19937 rng) {
83     // 0. Init : random generator, weights : array and already in
84     // 1. Initialize the random length (from 10 to 30) of array with r
85     // 2. uniform_int_distribution into range(10, 30);
86     // 3. N = range(rng);
87     // 4. N = drawInt(rng, 10, 30); // This is because the uniform_in
88     // 5. // This must be the second one.
89     // 6. uniform_int_distribution into shuffleRange(1, 100);
90     // 7. int K_shuffle = shuffleRange(rng);
91     // 8. // int K_shuffle = drawInt(rng, 1, 100); // This is because the
92     // 9. // This must be out of the loop....?
93     // 10. uniform_int_distribution into weightRange(1, 100);
94     // 11. for (int i = 0; i < N; i++) {
95     // 12.     weights[i] = weightRange(rng);
96     // 13.     //weights[i] = drawInt(rng, 1, 100); // This is because th
97     // 14. }
98     // 15. for (int i = 0; i < N; i++) {
99     // 16.     weights[i] = -1; // Fill remaining with -1 for clarity
100     // 17. }
101     // 18. // Shuffle weights, N, K_shuffle, rng, false);
102     // 19. //
103     // 20. // TODO 2: Implement mergeSortCount
104     // 21. //
105     // 22. // Merge Sort / Merge

```

Q. The following is a sequence of 21 parts arriving in shuffled order. What is the minimum number of adjacent swaps to sort the sequence in ascending order?

63 39 65 55 90 63 43 97 86 85 89 39 72 43 38 44 82 85 28 6 48

A. 126

[Execution complete with exit code 0]

```

167 170     } else {
168 171         container[containerIndex++] = arr[rightIndex++];
169 172     }
170 173     // Lets suppose 1 7 9 10 , 2 3 4 5
171 174     // Left index will be fixed to 1 until right index goes to the end for right index
172 175     // then 2 will pass 7 9 10 => (mid - leftIndex + 1)
173 176     inv_num += (mid - leftIndex + 1); // Count inversions: all remaining elements in left half are
174 177     // greater than arr[rightIndex]
175 178     }
176 179     }
177 180     }

```

C How the Solution Was Reflected in the Submitted Code

All Things are explained on A.

Problem 1 — Part 2

Machine Job Scheduler

A Problem Encountered or Difficult Part

This was also the application of Activity Selection Problem which recommend us to use the Greedy Algorithm and also learned in Algorithm Class.

B How You Solved It — Evidence & Process

The Key point of this implementation is “maxNonoverlappingjobs”

I've already known that we have to greedily choose the from smallest end job to largest, considering not overlapping/

So, I first Sort the job array. Because choosing from small to Large requires the array to be sorted already, I use just bubble sort. Basic Sort algorithm though requires $O(N^2)$, looking at the constraints the array list is at most 15 and this short case, other fascinating algorithm is kind of waste time and I did not feel to implement quicksort or higher version.

Now I got Job array sorted from small to large with respect to end time. Because the machine was used also in Prob1-1, It needs jobs starting before K to be rejected.

So, Start time would be K named as “last_end_time” and this variable will be updated whenever machine meets a job that starts after last_end_time with the end_time + R(rest time)

All process is manually calculated with seed40 and 666 and logics are established on that case

```

251 }
252
253 // =====
254 // TDOO & Implement maxNonOverlappingJobs
255 // =====
256
257 int maxNonOverlappingJobs(long long K, int R, Job jobs[], int M) {
258
259     // 1. Sort jobs by end time (Greedy approach for interval scheduling)
260     // bubble sort (under 20 -> it shows greater performance than other algorithms)
261     for (int i = 0; i < M; i++) {
262         for (int j = i + 1; j < M; j++) {
263             if (jobs[i].end > jobs[j].end) {
264                 swap(jobs[i], jobs[j]);
265             }
266             else if (jobs[i].end == jobs[j].end) {
267                 if (jobs[i].start > jobs[j].start) {
268                     swap(jobs[i], jobs[j]);
269                 }
270             }
271         }
272     }
273
274     // seed = 666
275     // 132 -> 138 -> 141 -> 149 -> 151 -> 154 -> 157 -> 160
276     int count = 0;
277     int last_end_time = K;
278     for (int i = 0; i < M; i++) {
279         if (jobs[i].start > last_end_time) {
280             count++;
281             last_end_time = jobs[i].end + R;
282         }
283     }
284
285     return count;
286 }

```

C How the Solution Was Reflected in the Submitted Code

Explain how this solution is present in your submitted code...

Explained in B

Problem 2

Matrix Equation Solver

A Problem Encountered or Difficult Part

First, it was the problem for me how to look at the Matrix.

To deal with the matrix in a row-wise way, it is a must to combine the One row in one struct, but allocate another row is waste of space and time. Thus, I decided to use “one of the greatest concepts of C or C++, Pointer to point the pointer”, which I will call `~~View`.

Second, after implement the `rowView`, as usually do, I did make a friend operator for `rowView` struct to deal with the computation more readably.

Here, Because we were using the struct that contain the pointer to point the matrix pointer, this temporary pointer is not adequate for left type, `RowView&`, which results compile error. Finally changed the friend to overloading...

Third, I somehow ignore the instruction and doing “my Gaussian Elimination” and reached that the final form of the matrix looking like Upper Triangular which made additional computation so, I decided to follow the instruction

B How You Solved It — Evidence & Process

All process is carved on my github

https://github.com/SongWooYoung/EEP_PJ2

1. Make Matrix view that refers to matrix pointer
2. Make RowView that refers to specific row with operator overloading ([,])
3. Normailize -> focus one the pivot line and make its line to start with 1

1 0 -2 7

0 1 3 -2

0 0 1 -2

4. Delete its upper value from diagonal to zero

1 0 0 3

0 1 0 4

0 0 1 -2

C How the Solution Was Reflected in the Submitted Code

Explain how this solution is present in your submitted code...

```
struct RowView {
    double* row;
    int width;

    // Watch only row array

    // Hadamard subtraction
    RowView operator-(const RowView& right) {
        for (int i = 0; i < width; i++) {
            row[i] -= right.row[i];
        }
        return *this;
    }

    // scalar division
    RowView operator/(const double& value) {
        for (int i = 0; i < width; i++) {
            row[i] /= value;
        }
        return *this;
    }

    //Row_i = Row_i - (factor * Row_pivot)
    void subtractScaleRow(const RowView& pivotRow, double factor) {
        for (int i = 0; i < width; i++) {
            row[i] -= factor * pivotRow.row[i];
        }
    }
};

struct MatrixView {
    int n;
    double (*matrix)[MAX_N + 1];

    // Focus on row array
    RowView operator[](int index) {
        return RowView(matrix[index], n + 1);
    }

    friend ostream& operator<<(ostream& os, const MatrixView& view) {
        for (int i = 0; i < view.n; i++) {
            for (int j = 0; j < view.n + 1; j++) {
                os << view.matrix[i][j] << " ";
            }
            os << '\n';
        }
        return os;
    }
};
```

All things are spiecified in A, B

Section 4

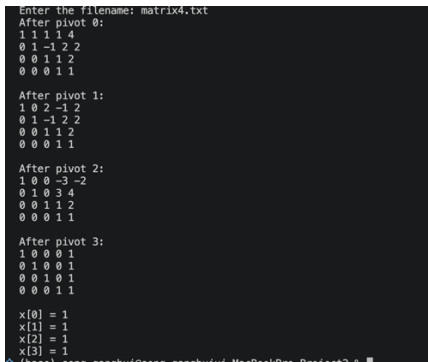
Code Logic Explanation

Problem 1 (seed = 666) — Part 2: Machine Job Scheduler**Target question:**

Q. What is the maximum number of non-overlapping jobs with start_time $\geq K$?

Value of K	<i>State the value of K generated with seed = 666</i> 35
Value of R	<i>State the value of R</i> 1
Jobs selected	<i>List all jobs selected for the optimal solution</i> 6, 1, 3
Selection logic	<i>Explain the logical process</i> Greedy Algorithm (after Sorting with respect to end time in ascending order)

Problem 2 (matrix2.txt) — Matrix Equation Solver

Initial augmented matrix (Screen shot)	<i>State or show the 2x2 augmented matrix from matrix2.txt</i>  <pre> Enter the filename: matrix2.txt After pivot 0: 1 1 1 4 0 1 -1 2 2 0 0 1 1 2 0 0 0 1 1 After pivot 1: 1 0 2 -1 2 0 1 -1 2 2 0 0 1 1 2 0 0 0 1 1 After pivot 2: 1 0 0 -3 -2 0 1 0 3 4 0 0 1 1 2 0 0 0 1 1 After pivot 3: 1 0 0 1 0 1 0 1 0 0 1 0 1 0 0 0 1 1 x[0] = 1 x[1] = 1 x[2] = 1 x[3] = 1 </pre>
Logical process used to obtain the solution	1. Get n-th Row 2. Norm : converts diagonal value to 1 3. Eliminate : Row_t -= Row_pivot * factor